입출력 포트와 LED

실습 2_1–2_4 전체 코드 · 동작 설명 · 암기 포인트

자료 작성 기준

교재의 포인터 접근 방식, 레지스터 사용 방식, 실행 순서와 지연함수 내부를 유지했다. 빈칸과 생략된 출력문을 채우고 문장 끝의 세미콜론을 추가했다. 별도의 반복문이나 다른 지연함수로 대체하지 않았다.

원문과 보충 구분

- 2_1: 교재 코드를 유지했다.
- 2_2: ??를 데이터시트 주소와 출력값으로 채웠다.
- 2_3: …에 PB2–PB6 출력문을 채우고 세미콜론을 추가했다. 원문의 Ox는 16진수 표기인 0x로 수정했다.
- 2_4: 함수 선언과 각 출력 다음의 함수 호출을 추가했다. **인수 1000은 교재에 없는 예시값**이며, 수업에서 지정한 값이 있으면 그 값을 따른다.

먼저 기억할 세 가지

DDRx: 각 비트가 1이면 출력, 0이면 입력이다.
PORTx: 출력으로 설정한 핀에 내보낼 값을 쓴다.
이 실습의 연결: 출력 1이면 LED가 켜지고, 0이면 꺼지는 연결을 기준으로 한다.

출처와 검증 범위

「마이크로프로세서_실습_02주차_입출력포트_LED_v01_260909」 11, 16–20쪽 및 첨부 ATmega128L 데이터시트의 Register Summary를 참고했다. 실습 2_2는 교재가 안내한 데이터시트 363쪽의 레지스터 주소를 기준으로 한다.

이 자료는 교재 학습용이다. AVR 컴파일 및 실제 보드 동작은 검증하지 않았다. 실제 지연시간은 클록과 최적화 설정에 따라 달라진다.

실습 2_1 · PORT_A로 모두 켜기

교재 17쪽 · 원문 코드 유지

```
int main(void)
{
    *(char *)0x3A = 0xff;   // 포트 A를 출력 포트로 사용
    *(char *)0x3B = 0xff;   // 0xff = 0b11111111, LED 모두 ON
}
```

코드 읽기

코드	의미
(char *)0x3A	데이터 메모리 주소 0x3A를 char형 포인터로 해석한다.
*(char *)0x3A = 0xff;	DDRA에 0xff를 써서 PA7–PA0을 모두 출력으로 설정한다.
*(char *)0x3B = 0xff;	PORTA에 0xff를 써서 모든 핀에 1을 출력한다.

왜 LED가 모두 켜질까?

0xff = 0b11111111이다. 여덟 비트가 모두 1이므로 여덟 LED가 모두 켜진다. 교재의 JKIT-128-1 LED는 PORT_A에 연결되며, 왼쪽은 PA7, 오른쪽은 PA0이다.

시험 포인트

- 첫 줄은 방향 설정, 두 번째 줄은 출력값 설정이다.
- *는 해당 주소의 대상에 접근하는 역참조 연산자이다.
- DDRA = 0x3A, PORTA = 0x3B를 한 쌍으로 암기한다.

> 교재대로 main 안에 두 문장만 있다. 임의로 while문이나 다른 레지스터 표현을 추가하지 않았다.

실습 2_2 · PORT_B로 모두 켜기

교재 18쪽 · 빈칸 보충

```
int main(void)
{
    *(char *)0x37 = 0xff;   // 포트 B를 출력 포트로 사용
    *(char *)0x38 = 0xff;   // 0xff = 0b11111111, LED 모두 ON
}
```

빈칸의 정답

역할	레지스터	I/O 주소	데이터 메모리 주소
출력 방향 설정	DDRB	0x17	**0x37**
출력값 설정	PORTB	0x18	**0x38**

데이터시트의 $17 ($37), $18 ($38)에서 괄호 안의 값이 데이터 메모리 주소다. 이 코드처럼 C 포인터로 접근할 때는 괄호 안 주소를 사용한다. 해당 I/O 영역에서는 데이터 메모리 주소가 I/O 주소보다 0x20 크다.

실행 순서

1. DDRB에 0xff를 써서 PB7–PB0을 모두 출력으로 설정한다.
2. PORTB에 0xff를 써서 모든 PB 핀에 1을 출력한다.

회로와 함께 기억하기

교재는 Bread board, LED, 저항을 이용하도록 안내한다. 보드의 PORT_A LED를 그대로 두고 PORT_B에 값을 쓴다고 해서 PORT_A LED가 제어되지는 않는다. PORT_B에 실습 회로를 연결해야 한다.

> 주소와 값만 채웠다. 교재의 (char *) 및 * 기호와 문장 구조는 유지했다.

실습 2_3 · 소등 위치를 순서대로 변경

교재 19쪽 · 생략된 패턴과 문법 보충

```
#include <avr/io.h>

int main(void)
{
    DDRB = 0xFF;             // 포트 B를 모두 출력 포트로 설정
    while(1)
    {
        PORTB = 0xFE; // PB0만 소등
        PORTB = 0xFD; // PB1만 소등
        PORTB = 0xFB; // PB2만 소등
        PORTB = 0xF7; // PB3만 소등
        PORTB = 0xEF; // PB4만 소등
        PORTB = 0xDF; // PB5만 소등
        PORTB = 0xBF; // PB6만 소등
        PORTB = 0x7F; // PB7만 소등
    }
}
```

코드 읽기

<avr/io.h>는 DDRB, PORTB 같은 AVR 레지스터 이름을 제공한다. DDRB = 0xFF는 모든 PB 핀을 출력으로 설정한다. while(1)은 중괄호 안의 여덟 출력문을 계속 반복한다.

꺼지는 LED	출력값	PB7 → PB0
PB0	0xFE	11111110
PB1	0xFD	11111101
PB2	0xFB	11111011
PB3	0xF7	11110111
PB4	0xEF	11101111
PB5	0xDF	11011111
PB6	0xBF	10111111
PB7	0x7F	01111111

0인 비트 하나만 오른쪽에서 왼쪽으로 이동한다. 각 패턴에서는 해당 LED 하나만 꺼지고 나머지 일곱 개는 켜진다.

> 시간 지연이 없어 출력이 매우 빠르게 바뀐다. 눈으로 단계별 변화를 구분하기 어렵기 때문에 다음 실습에서 지연함수를 넣는다.

실습 2_4 · 출력 사이에 시간 지연 추가

교재 20쪽 · 전체 코드 / **1000은 교재에 없는 예시 인수**

```
#include <avr/io.h>

void msec_delay(int n);

int main(void)
{
    DDRB = 0xFF;             // 포트 B를 모두 출력 포트로 설정
    while(1)
    {
        PORTB = 0xFE; // PB0만 소등
        msec_delay(1000);
        PORTB = 0xFD; // PB1만 소등
        msec_delay(1000);
        PORTB = 0xFB; // PB2만 소등
        msec_delay(1000);
        PORTB = 0xF7; // PB3만 소등
        msec_delay(1000);
        PORTB = 0xEF; // PB4만 소등
        msec_delay(1000);
        PORTB = 0xDF; // PB5만 소등
        msec_delay(1000);
        PORTB = 0xBF; // PB6만 소등
        msec_delay(1000);
        PORTB = 0x7F; // PB7만 소등
        msec_delay(1000);
    }
}

// 시간 지연함수 추가
void msec_delay(int n)
{
    int i;

    for (; n > 0; n--)          // 1msec 루프를 n회 반복
    {
        for (i = 0; i < 2000; i++)   // 1msec 시간 지연 루프
        {
            asm("nop"); asm("nop");
        }
    }
}
```

msec_delay의 내부는 교재 그대로다. 함수 선언은 main에서 뒤에 정의된 함수를 호출하기 위해 추가했다. 각 출력 다음에 지연 호출을 넣어 마지막 PB7 패턴도 유지한 뒤 PB0으로 돌아가게 했다.

지연함수 해설과 시험 직전 확인

msec_delay(int n)의 흐름

1. 호출할 때 전달한 값이 n에 들어간다.
2. 바깥 for문은 n이 0보다 클 동안 반복하며, 반복이 끝날 때마다 n을 1 줄인다.
3. 안쪽 for문은 i = 0부터 i = 1999까지 총 2000번 반복한다.
4. 각 안쪽 반복에서 asm("nop");를 두 번 실행한다. nop은 명시적인 동작을 하지 않는 CPU 명령이다.

예시의 n = 1000이면 바깥 반복은 1000번, 안쪽 반복은 총 2,000,000번이다. 소스에 적힌 nop은 총 4,000,000번 실행되는 구조다. 루프 제어에도 시간이 걸리므로 이 횟수만으로 정확한 지연시간을 계산하면 안 된다.

> **시간 해석 주의:** 교재 주석은 안쪽 반복을 1msec 지연으로 설명한다. 이 가정을 따르면 msec_delay(1000)은 약 1초를 의도하지만, 실제 시간은 CPU 클록과 컴파일 옵션을 확인해야 한다.

시험 직전 핵심 확인

질문	답
모든 핀을 출력으로 설정하는 값은?	DDRx에 0xFF
모든 LED를 켜는 출력값은?	이 실습의 연결 기준 PORTx에 0xFF
PB0만 끄려면?	PORTB = 0xFE;
PB7만 끄려면?	PORTB = 0x7F;
2_1과 2_2의 차이는?	제어 포트와 두 레지스터의 주소
2_3과 2_4의 차이는?	각 출력 후 지연함수를 호출하는지 여부
0xFF와 0xff의 값은 같은가?	같다. 둘 다 255이며 8비트 모두 1이다.

교재 표기와 구분해 기억하기

16진수 접두사는 숫자 0을 쓴 0x다. 문장 끝에는 세미콜론이 필요하다. 교재의 …는 실제 C 코드가 아니라 생략 표시이므로 PB2~PB6 문장으로 채웠다. 2_4의 1000은 수업 지정값이 아니라 자료 작성을 위한 예시다.

다른 주차도 같은 형식으로 정리한 뒤 주차 순서대로 병합할 수 있도록 구성했다.

8×8 LED 매트릭스와 스위치 입력

실습별 전체 코드 · 입력 대기 · 풀업 · 디바운싱

번호와 원문 유지 기준

업로드 파일은 03주차지만 본문은 실습 #2 및 2_1–2_4로 표기되어 있다. 이 자료는 주차를 3주차로, 실습 번호는 원문 그대로 표시한다. 2주차 자료의 동명 실습과 구분해야 한다.

배열 이름, if(++i==8) i=0;, 비트 연산 기호와 대기 while문의 세미콜론을 유지했다. 빈칸을 채우고, 교재의 매트릭스 코드와 이미지 속 스위치 참고 코드를 조합했다. 후반 실습의 전체 프로그램은 교재에 그대로 인쇄된 원문이 아니라 아래 지시에 따른 보충본이다.

보충·선택 사항

- 배열: 7쪽 회로의 행 애노드·열 캐소드 연결을 기준으로 PORTA는 한 비트만 1, PORTD는 모두 0으로 채웠다.
- 입력: 11쪽 지시대로 참고 코드의 PD0/PIND 대신 PE0/PINE을 사용하고, PORTD는 매트릭스 출력으로 유지했다.
- 2_3_1: 내부 풀업 비활성화를 명확히 하기 위해 PORTE &= 0xFE;를 추가했다. 외부 풀업 제거는 회로에서 한다.
- 2_4: 직전 실습의 내부 풀업을 유지하고, 25쪽 순서대로 누름·뗌 뒤에 지연을 추가했다.
- 매크로명: 22쪽의 DEBOUNDING_DELAY를 사용했다. 26쪽 참고 코드는 DEBOUNCING_DELAY로 표기가 다르므로 혼용하지 않는다.

실행 전 설정

대상 MCU는 ATmega128 계열이다. <util/delay.h>의 계산에 쓰이는 F_CPU를 실제 CPU 클록과 일치하도록 프로젝트 설정에 지정하고 최적화를 활성화한다. 교재에 없는 클록 수치를 임의로 코드에 추가하지 않았다.

코드는 독립된 실습 파일로 사용한다. AVR 컴파일과 실제 회로 동작은 확인하지 않았다. 첨부 데이터시트의 GPIO 및 PUD 설명을 대조했다.

회로와 패턴 배열 읽기

교재 7–8쪽 기준 · 매트릭스 부품의 행/열 극성이 같아야 한다.

연결과 점등 조건

신호	연결	점등 시 상태
PA0–PA7	ROW 1–8	선택한 행만 HIGH(1)
PD0–PD7	COL 1–8	모든 열 LOW(0)
PE0	스위치 입력	뗌 HIGH, 누름 LOW

행에서 열 방향으로 전류가 흐르는 교재 회로를 기준으로, 한 행만 1로 만들고 열을 모두 0으로 하면 그 행의 LED 여덟 개가 켜진다. 매트릭스 실물 핀 번호와 행/열 번호는 다르므로 교재 배선도를 따른다.

```
unsigned char pattern_port_A[8]
= { 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80 };

unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };
```

배열 인덱스	행	PORTA	PORTD
0	1행	0x01	0x00
1	2행	0x02	0x00
2	3행	0x04	0x00
3	4행	0x08	0x00
4	5행	0x10	0x00
5	6행	0x20	0x00
6	7행	0x40	0x00
7	8행	0x80	0x00

2주차의 LED 하나씩 소등하는 0xFE, 0xFD… 패턴을 그대로 복사하는 실습이 아니다. 여기서는 교재 7쪽의 매트릭스 극성과 8쪽의 “한 행 점등” 목표에 맞추어 배열의 빈칸을 채운다.

실습 2_1 · 한 행씩 순차 점등

교재 8쪽 전체 코드 · 배열 빈칸 보충

```
#include <avr/io.h>
#include <util/delay.h>

void msec_delay(int n)
{
    for(; n >0; n--)
        _delay_ms(1);
}

unsigned char pattern_port_A[8]
= { 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80 };

unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

int main(void)
{
    DDRA = 0xff;
    DDRD = 0xFF;
    int i=0;

    while (1)
    {
        if(++i==8) i=0;
        PORTA = pattern_port_A[i];
        PORTD = pattern_port_D[i];
        msec_delay(1000);
    }
}
```

실행 흐름과 주의점

두 포트를 출력으로 설정한 뒤 인덱스를 증가시키고, 같은 인덱스의 행·열 패턴을 출력한다. msec_delay(1000)은 _delay_ms(1)을 1000번 호출한다.

시작 행: 원문은 i=0 다음에 ++i를 먼저 실행한다. 따라서 첫 명시적 출력은 인덱스 1, 즉 2행이다. 이후 3→4→5→6→7→8→1행으로 반복한다. 교재의 “1행부터” 설명과 차이가 있지만 실행 순서를 바꾸지 않았다.

실습 2_2 · 외부 입력으로 제어

교재 10-11쪽 · 매트릭스 기본 코드에 입력 대기 조합

```
#include <avr/io.h>
#include <util/delay.h>

void msec_delay(int n)
{
    for(; n >0; n--)
        _delay_ms(1);
}

unsigned char pattern_port_A[8]
= { 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80 };

unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

int main(void)
{
    int i=0;
    DDRA = 0xff;
    DDRD = 0xFF;
    DDRE = 0x00;
    PORTA = pattern_port_A[i];
    PORTD = pattern_port_D[i];

    while (1)
    {
        while(!(~PINE & 0x01));
        if(++i==8) i=0;
        PORTA = pattern_port_A[i];
        PORTD = pattern_port_D[i];
        while(~PINE & 0x01);
    }
}
```

PE0에 외부 풀업 저항과 GND로 연결되는 스위치를 사용한다. 초기에는 인덱스 0의 1행을 출력하고, 누르면 다음 행으로 바뀐다. 버튼을 계속 누르는 동안에는 뗌 대기에서 멈춘다. 아직 디바운싱 지연은 없다.

실습 2_3_1 · 플로팅 확인

교재 14–15쪽 · 외부 풀업을 제거한 회로에서 실행

```
#include <avr/io.h>
#include <util/delay.h>

void msec_delay(int n)
{
    for(; n >0; n--)
        _delay_ms(1);
}

unsigned char pattern_port_A[8]
= { 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80 };

unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

int main(void)
{
    int i=0;
    DDRA = 0xff;
    DDRD = 0xFF;
    DDRE = 0x00;
    PORTE &= 0xFE; // PE0 내부 풀업을 사용하지 않음
    PORTA = pattern_port_A[i];
    PORTD = pattern_port_D[i];

    while (1)
    {
        while(!(~PINE & 0x01));
        if(++i==8) i=0;
        PORTA = pattern_port_A[i];
        PORTD = pattern_port_D[i];
        while(~PINE & 0x01);
    }
}
```

PORTE의 비트 0을 0으로 만들어 내부 풀업도 사용하지 않는다. 스위치를 떼면 PE0가 HIGH 또는 LOW로 확정되지 않아 의도하지 않은 패턴 이동이 발생할 수 있다. 반드시 매번 오동작하는 것은 아니다. 외부 풀업이 남아 있으면 플로팅 실험이 되지 않는다.

실습 2_3_2 · 내부 풀업 사용

교재 17–20쪽 · 내부 풀업 세 조건 적용

```
#include <avr/io.h>
#include <util/delay.h>

void msec_delay(int n)
{
    for(; n >0; n--)
        _delay_ms(1);
}

unsigned char pattern_port_A[8]
= { 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80 };

unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

int main(void)
{
    int i=0;
    DDRA = 0xff;
    DDRD = 0xFF;
    DDRE &= 0xFE; // PE0 입력 설정
    SFIOR &= 0xFB; // 내부 풀업 사용 허용
    PORTE |= 0x01; // PE0 내부 풀업 활성화
    PORTA = pattern_port_A[i];
    PORTD = pattern_port_D[i];

    while (1)
    {
        while(!(~PINE & 0x01));
        if(++i==8) i=0;
        PORTA = pattern_port_A[i];
        PORTD = pattern_port_D[i];
        while(~PINE & 0x01);
    }
}
```

DDRE의 비트 0 = 0, SFIOR의 PUD(비트 2) = 0, PORTE의 비트 0 = 1을 모두 만족한다. 외부 풀업 없이도 버튼을 떼면 HIGH를 유지한다. 풀업은 플로팅을 줄이며 기계적 채터링을 제거하는 기능은 아니다.

실습 2_4 · 디바운싱 지연

교재 22, 25–26쪽 · 내부 풀업 유지 / 매크로 표기는 22쪽 기준

```
#include <avr/io.h>
#include <util/delay.h>

#define DEBOUNDING_DELAY 20

void msec_delay(int n)
{
    for(; n >0; n--)
        _delay_ms(1);
}

unsigned char pattern_port_A[8]
= { 0x01, 0x02, 0x04, 0x08, 0x10, 0x20, 0x40, 0x80 };

unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 };

int main(void)
{
    int i=0;
    DDRA = 0xff;
    DDRD = 0xFF;
    DDRE &= 0xFE; // PE0 입력 설정
    SFIOR &= 0xFB; // 내부 풀업 사용 허용
    PORTE |= 0x01; // PE0 내부 풀업 활성화
    PORTA = pattern_port_A[i];
    PORTD = pattern_port_D[i];

    while (1)
    {
        while(!(~PINE & 0x01));
        msec_delay(DEBOUNDING_DELAY);
        if(++i==8) i=0;
        PORTA = pattern_port_A[i];
        PORTD = pattern_port_D[i];
        while(~PINE & 0x01);
        msec_delay(DEBOUNDING_DELAY);
    }
}
```

누름 감지 → 20ms 지연 → 패턴 변경 → 뗌 감지 → 20ms 지연 순서다. 교재 방식대로 지연 뒤 재검사 조건은 추가하지 않았다. 이 방식의 100% 안정성을 코드만으로 보장할 수 없으므로 실제 스위치에서 확인한다.

대기 조건과 비트 연산 해설

두 while문은 무엇을 기다릴까?

```
while(!(~PINE & 0x01)); // 누름 대기
while(~PINE & 0x01);    // 뗌 대기
```

스위치	PE0	~PINE & 0x01	!(~PINE & 0x01)
뗌	1	0	1
누름	0	1	0

누름 대기는 버튼을 떼고 있을 때 조건이 1이므로 계속 반복한다. 누르면 조건이 0이 되어 다음 문장으로 진행한다. 뗌 대기는 버튼을 누르고 있을 때 조건이 1이며, 떼면 종료된다.

세미콜론을 지우면 안 된다. 이 while문 뒤의 ;는 빈 반복문 본문이다. 지우면 다음 문장이 반복문 본문으로 묶여 실행 의미가 달라진다. ~, !, &, 괄호도 교재 그대로 유지한다.

내부 풀업 세 줄

```
    DDRE &= 0xFE; // PE0 입력 설정
    SFIOR &= 0xFB; // 내부 풀업 사용 허용
    PORTE |= 0x01; // PE0 내부 풀업 활성화
```

마스크	2진수	효과
0xFE	11111110	&=로 비트 0만 0으로 만든다.
0xFB	11111011	&=로 비트 2만 0으로 만든다.
0x01	00000001	|=로 비트 0만 1로 만든다.

&=와 |=는 해당 비트를 바꾸면서 다른 비트의 기존 값을 보존한다. PUD는 Pull-Up Disable이므로 1이면 풀업 금지, 0이면 사용 허용이다.

시험 직전 비교와 확인 문제

실습별 차이

실습	패턴 변경 조건	입력 안정화
2_1	1000ms 지연마다 자동 이동	입력 사용 안 함
2_2	PE0 누름·뗌으로 이동	외부 풀업
2_3_1	같은 입력 코드	풀업 제거 → 플로팅 관찰
2_3_2	같은 입력 코드	내부 풀업
2_4	누름 후 지연하고 이동	내부 풀업 + 누름·뗌 뒤 지연

자주 묻는 질문

왜 PORT가 아니라 PIN을 읽나?
PINE은 입력 핀의 논리 상태를 읽는다. PORTE는 출력값 또는 입력 모드의 풀업 설정에 사용한다.

if(++i==8) i=0;의 역할은?
i를 먼저 1 증가시키고 8이면 0으로 되돌린다. 배열 인덱스를 0–7 범위로 순환시킨다.

풀업과 디바운싱은 같은가?
풀업은 열린 입력의 기본 전압을 정한다. 디바운싱은 스위치 접점이 여러 번 붙었다 떨어지는 현상을 다룬다.

DEBOUNDING_DELAY는 함수형 매크로인가?
#define DEBOUNDING_DELAY 20은 객체형 매크로인 상수 정의다. 교재는 “매크로 함수”라는 표현도 쓰지만, 이 정의에는 매개변수가 없다. 사용 위치에서 20으로 치환되며 끝에 세미콜론을 붙이지 않는다.

20ms를 늘리면 항상 좋은가?
채터링 억제에는 도움이 될 수 있으나 입력 반응이 느려질 수 있다. 교재의 20ms를 기준으로 실제 회로에서 확인한다.

원문 출처: 업로드한 03주차 PDF 7–8, 10–11, 14–15, 17–20, 22–26쪽. 보충 코드는 해당 지시를 연결한 학습용 전체 코드이며 교재의 완성 원문과 동일하다고 단정하지 않는다. 실제 회로·컴파일 검증은 미실시다.

외부 인터럽트 INT4와 LED 제어

실습 3_1–3_3 · 배포 코드 빈칸 완성 · 시험 대비 해설

기준 자료와 코드 유지 원칙

4주차 PDF와 실습코드 #3-1, #3-2, #3-3 배포 TXT를 함께 대조했다. 배포 TXT의 기존 문장·연산자·주석·실행 순서는 유지하고 빈칸을 채웠다. 들여쓰기와 빈 줄은 읽기 편하게 정리했다. 각 실습은 별도 C 파일이다.

무엇을 채웠나?

- 공통: INT4_vect, EICRB, ISC41, sei를 채웠다.
- 3_1·3_2: ISR의 int i 앞에 static을 채웠다.
- 3_3: 전역 int index 앞에 volatile을 채우고 main의 두 출력문을 채웠다.
- 3_2·3_3: 두 지연 호출과 EIFR = (1<<INTF4);를 채웠다.
- 배열: 3_3은 PDF 9쪽의 명시된 값을 사용했다. 3_1·3_2는 PDF와 TXT 모두 값이 비어 있어, 비교 학습을 위해 3_3의 배열로 통일해 보충했다. 이 두 배열은 교재의 유일한 정답으로 확정된 값이 아니다.

배선과 실행 전제

스위치 입력은 **PE4 = INT4**이다. 3주차에서 사용한 PE0와 다르다. 스위치를 떼면 HIGH, 누르면 LOW가 되도록 외부 풀업 및 GND 연결을 사용한다. 배포 코드에는 PE4 내부 풀업을 켜는 코드가 없으므로 임의로 추가하지 않았다.

EICRB |= (1<<ISC41);는 리셋 직후 ISC40 = 0인 상태를 전제로 하강 모서리를 선택한다. 다른 코드에서 ISC40을 1로 바꾼 상태라면 이 한 줄만으로 10이 되지는 않는다.

<util/delay.h>의 F_CPU는 실제 CPU 클록에 맞게 프로젝트에서 지정하고 최적화를 활성화한다. 클록 수치는 임의로 코드에 넣지 않았다. 자료의 코드에 대해 AVR 컴파일 및 보드 동작 검증은 하지 않았다.

실습 3_1 · 인터럽트를 이용한 LED 제어

배포 TXT #3-1 기준 · 전체 코드

```
// 실습 3_1. 인터럽트를 이용한 LED 제어

#include <avr/io.h>// I/O 레지스터 정의
#include <avr/interrupt.h>// 인터럽트 정의
#include <util/delay.h>// 시간 지연 함수 헤더파일

#define DEBOUNDING_DELAY 20

void msec_delay(int n)
{
    for(; n >0; n--)
    _delay_ms(1);
}

unsigned char pattern_port_A[8]
= { 0x80, 0x04, 0x00, 0x10, 0x00, 0x00, 0x00, 0x00 };
unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x80, 0x00, 0x01, 0x40, 0x02, 0x10 };

ISR (INT4_vect)
{
    static int i = 0;// 패턴 인덱스

    if(++i==8) i=0;// 증가 값이 8이면 0으로 리셋
    PORTA = pattern_port_A[i];// i-번째 패턴으로 LED를 켠다.
    PORTD = pattern_port_D[i];
}

int main(void)
{
    DDRA = 0xff;
    DDRD = 0xff;

    PORTA = pattern_port_A[0];// 포트 A를 출력 포트로 설정
    PORTD = pattern_port_D[0];// 처음 패턴으로 LED를 켠다.
    EICRB |= (1<<ISC41);// INT4: 하강모서리 트리거 (ISC41:ISC40=10)
    EIMSK |= (1<<INT4); // INT4 허용
    sei(); // 전역 인터럽트 허용

    while(1);// 무한루프
}
```

배포 코드의 빈칸을 채운 전체 코드다. 지연함수와 매크로는 배포본 그대로 남겼지만 이 실습에서는 호출하지 않는다. ISR의 static i는 이전 호출의 값을 유지한다.

실습 3_2 · 인터럽트와 디바운싱

배포 TXT #3-2 기준 · 전체 코드

```
// 실습 3_2. 인터럽트와 디바운싱을 이용한 LED 제어

#include <avr/io.h>// I/O 레지스터 정의
#include <avr/interrupt.h>// 인터럽트 정의
#include <util/delay.h>// 시간 지연 함수 헤더파일

#define DEBOUNDING_DELAY 20

void msec_delay(int n)
{
    for(; n >0; n--)
    _delay_ms(1);
}

unsigned char pattern_port_A[8]
= { 0x80, 0x04, 0x00, 0x10, 0x00, 0x00, 0x00, 0x00 };
unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x80, 0x00, 0x01, 0x40, 0x02, 0x10 };

ISR (INT4_vect)
{
    static int i = 0;// 패턴 인덱스

    if(++i==8) i=0;// 증가 값이 8이면 0으로 리셋
    PORTA = pattern_port_A[i];// i-번째 패턴으로 LED를 켠다.
    PORTD = pattern_port_D[i];
    msec_delay(DEBOUNDING_DELAY);// 스위치 누름에 대한 디바운싱
    while(~PINE & 0x10);// 스위치 해제를 기다림
    msec_delay(DEBOUNDING_DELAY);// 스위치 뗌에 대한 디바운싱
    EIFR = (1<<INTF4);// INT4 인터럽트 플래그 클리어
}

int main(void)
{
    DDRA = 0xff;
    DDRD = 0xff;

    PORTA = pattern_port_A[0];// 포트 A를 출력 포트로 설정
    PORTD = pattern_port_D[0];// 처음 패턴으로 LED를 켠다.
    EICRB |= (1<<ISC41);// INT4: 하강모서리 트리거 (ISC41:ISC40=10)
    EIMSK |= (1<<INT4); // INT4 허용
    sei(); // 전역 인터럽트 허용

    while(1);// 무한루프
}
```

출력 변경 → 누름 지연 → 뗌 대기 → 뗌 지연 → 플래그 클리어 순서를 유지했다. 기존 while문의 ~, &, 0x10, 세미콜론을 변경하지 않았다.

실습 3_3 · 메인 루프에서 패턴 출력

배포 TXT #3-3 기준 · 전체 코드

```
// 실습 3_3. 디바운싱과 인터럽트 기반 LED 제어 및 메인 루프에서 패턴 출력

#include <avr/io.h>// I/O 레지스터 정의
#include <avr/interrupt.h>// 인터럽트 정의
#include <util/delay.h>// 시간 지연 함수 헤더파일

#define DEBOUNDING_DELAY 20

void msec_delay(int n)
{
    for(; n >0; n--)
    _delay_ms(1);
}
volatile int index = 0;// 패턴 인덱스

unsigned char pattern_port_A[8]
= { 0x80, 0x04, 0x00, 0x10, 0x00, 0x00, 0x00, 0x00 };
unsigned char pattern_port_D[8]
= { 0x00, 0x00, 0x80, 0x00, 0x01, 0x40, 0x02, 0x10 };

ISR (INT4_vect)
{
    if(++index==8) index=0;// 증가 값이 8이면 0으로 리셋
    msec_delay(DEBOUNDING_DELAY);// 스위치 누름에 대한 디바운싱

    while(~PINE & 0x10);// 스위치 해제를 기다림
    msec_delay(DEBOUNDING_DELAY);// 스위치 뗌에 대한 디바운싱
    EIFR = (1<<INTF4);// INT4 인터럽트 플래그 클리어
}

int main(void)
{
    DDRA = 0xff;
    DDRD = 0xff;

    PORTA = pattern_port_A[0];// 포트 A를 출력 포트로 설정
    PORTD = pattern_port_D[0];// 처음 패턴으로 LED를 켠다.
    EICRB |= (1<<ISC41);// INT4: 하강모서리 트리거 (ISC41:ISC40=10)
    EIMSK |= (1<<INT4); // INT4 허용
    sei(); // 전역 인터럽트 허용

    while(1) // 무한루프
    {
        PORTA = pattern_port_A[index];
        PORTD = pattern_port_D[index];
    }
}
```

ISR은 인덱스만 바꾸고, PORTA와 PORTD 출력은 main에서 한다. ISR이 버튼 뗌과 지연을 기다리는 동안 main은 실행되지 않으므로, 새 패턴은 ISR 복귀 뒤 출력된다.

인터럽트 설정과 플래그 해설

INT4가 실행되기 위한 세 단계

```
EICRB |= (1<<ISC41); // 리셋 상태 기준 ISC41:ISC40 = 10
EIMSK |= (1<<INT4);  // INT4 개별 허용
sei();              // 전역 인터럽트 허용
```

대상	역할
EICRB의 ISC41·ISC40	INT4를 어떤 입력 변화에서 발생시킬지 선택한다.
EIMSK의 INT4	INT4 요청을 허용한다.
SREG의 I 비트	전역 인터럽트를 허용한다. sei()로 1이 된다.
ISR(INT4_vect)	INT4에 연결된 인터럽트 서비스 루틴을 정의한다.

ISC41 : ISC40 조합

비트 조합	트리거
00	LOW 레벨
01	논리값 변화
10	하강 모서리: HIGH → LOW
11	상승 모서리: LOW → HIGH

왜 플래그에 1을 쓰나?

```
EIFR = (1<<INTF4);
```

EIFR의 INTF4는 1을 쓰면 지워지는 플래그다. INT4 ISR 진입 시 원래의 요청 플래그는 자동으로 지워진다. 하지만 ISR에서 기다리는 사이 채터링으로 새 요청이 쌓일 수 있어, 마지막에 다시 지운다.

> 일반 레지스터의 비트를 0으로 만드는 것과 다르다. EIFR에는 대상 비트에만 1을 쓰는 대입문을 사용했다. |=로 읽은 값을 다시 쓰면 다른 비트의 대기 플래그도 지울 수 있다.

근거: 첨부 ATmega128L 데이터시트 90–91쪽 EICRB, EIMSK, EIFR 및 PE4 대체 기능 설명.

static·volatile과 실습별 실행 흐름

3_1·3_2: static int i = 0;

ISR 안의 일반 지역 변수 int i = 0;는 호출될 때마다 0으로 초기화된다. 그러면 ++i는 매번 1이 되어 순차 이동하지 않는다. 배포본의 int 앞 빈칸에 static을 채우면 처음 한 번만 초기화되고 다음 인터럽트에도 이전 값이 유지된다.

초기 출력은 인덱스 0이다. 인터럽트마다 1 → 2 → ⋯ → 7 → 0으로 바뀐다. i는 ISR 내부에서만 사용하므로 이 코드에서는 공유 변수용 volatile이 필요하지 않다.

3_3: volatile int index = 0;

index는 ISR에서 바꾸고 main에서 읽는 전역 변수다. volatile은 컴파일러가 이 변수의 읽기를 불필요하다고 생략하거나 이전 값을 계속 재사용하지 않도록 한다. 빈칸의 int는 유지하며 앞에 volatile을 채웠다.

> volatile은 동시 접근의 원자성을 보장하지 않는다. 일반적인 AVR-GCC에서 int는 16비트이며 8비트 CPU의 읽기·쓰기는 여러 명령이 될 수 있다. 이 실습은 0–7로 제한된 인덱스와 교재 구조를 유지한다. 이를 임의의 16비트 공유 카운터에 그대로 일반화하면 안 된다.

3_2와 3_3의 출력 시점 차이

구분	실행 순서
3_2	ISR 진입 → 인덱스 증가 → 즉시 출력 → 지연 → 뗌 대기 → 지연 → 플래그 클리어 → 복귀
3_3	ISR 진입 → 인덱스 증가 → 지연 → 뗌 대기 → 지연 → 플래그 클리어 → 복귀 → main에서 출력

일반적인 ISR 실행 중에는 전역 인터럽트가 비활성화된다. 버튼을 길게 누르면 ISR의 while문에 계속 머물며 main도 멈춘다. 3_3에서 main과 ISR이 같은 순간에 병렬로 실행되는 것은 아니다.

누름 대기문은 왜 없나?

3주차는 main에서 누름 여부를 계속 검사했다. 4주차는 INT4의 하강 모서리 자체가 누름을 알린다. ISR 안에는 누름 대기 대신 뗌 대기가 남아 있다.

패턴 표와 시험 직전 확인

PDF 9쪽의 패턴값

인덱스	PORTA	PORTD
0	0x80	0x00
1	0x04	0x00
2	0x00	0x80
3	0x10	0x00
4	0x00	0x01
5	0x00	0x40
6	0x00	0x02
7	0x00	0x10

배열값은 PDF에 적힌 값을 그대로 옮겼다. 3주차의 한 행씩 켜는 배열로 바꾸지 않았다. 실제 점등 모양은 연결 회로와 LED 극성에 따라 달라지므로, 이 숫자만으로 “몇 행이 켜진다”고 단정하지 않는다.

빈칸 암기표

빈칸 역할	채울 내용
ISR 벡터	INT4_vect
INT4 제어 레지스터 / 비트	EICRB / ISC41
전역 인터럽트 허용 함수	sei
ISR 내부 인덱스 유지	static
main–ISR 공유 인덱스	volatile
누름·뗌 디바운싱	msec_delay(DEBOUNDING_DELAY)
플래그 클리어	EIFR = (1<<INTF4)

기호를 포함해 기억하기

while(~PINE & 0x10);는 PE4가 LOW인 동안 빈 반복문으로 기다린다. 0x10은 00010000이므로 비트 4만 검사한다. ;를 삭제하면 의미가 바뀐다.

if(++index==8) index=0;는 먼저 증가하고 8에서 0으로 돌아간다. ++와 ==를 구분한다. **DEBOUNDING_DELAY**의 철자는 배포본 그대로 유지했다.

실습 비교와 교재 주석 주의사항

실습별 차이

실습	인덱스	출력 위치	디바운싱
3_1	ISR 안 static i	ISR	없음
3_2	ISR 안 static i	ISR	누름·뗌 각 20ms
3_3	전역 volatile index	main	누름·뗌 각 20ms

배포 주석과 실제 역할

```
DDRA = 0xff;
PORTA = pattern_port_A[0];// 포트 A를 출력 포트로 설정
```

배포본의 PORTA 줄 주석은 그대로 보존했지만, 방향을 출력으로 설정하는 문장은 DDRA = 0xff;이다. PORTA 줄은 첫 번째 패턴의 출력값을 쓰는 역할이다. 시험에서 역할을 물으면 이 차이를 설명한다.

20ms면 항상 정확한가?

배포 코드에서는 패턴을 바꾼 뒤 누름 지연을 수행한다. 지연 뒤 신호를 다시 검증하는 조건은 없다. 20ms 지연은 채터링 영향을 줄이려는 방식이며, 모든 스위치와 잡음 환경에서 100% 정확함을 보장하지 않는다. 코드 진행을 바꾸지 않기 위해 별도 재검사 조건을 추가하지 않았다.

체크리스트

- 스위치가 PE0가 아니라 PE4에 연결되어 있는가?
- 버튼을 떼면 HIGH, 누르면 LOW인가?
- F_CPU와 실제 CPU 클록이 일치하는가?
- static과 volatile의 역할을 구분하는가?
- 3_3의 출력은 ISR 복귀 뒤라는 점을 아는가?
- EIFR의 1 쓰기 클리어를 일반 비트 클리어와 구분하는가?

확인 범위: 업로드한 배포 TXT 3개와 PDF 3·6·9쪽을 대조하고, 빈칸 채우기 후 기존 코드의 문자·기호가 삭제되지 않았는지 공백을 제외해 검사했다. 데이터시트로 INT4 설정과 플래그 동작을 확인했다. 실제 컴파일·보드 실행은 미실시다.

타이머/카운터0와 버저 연주

실습 4_1–4_4 · 전체 코드 · 계산식 · 시험 핵심

기준과 작성 원칙

5주차 PDF와 배포 TXT 4개를 대조했다. 실습 번호는 교재의 4_1–4_4를 유지한다. 기존 실행 순서, 연산자, 배열명과 주석은 보존하며 빈칸을 채웠다. 코드 파일은 실습별로 독립적으로 사용한다.

빈칸 외 최소 보완 — 모두 명시

- 4_1·4_2의 Static을 C 키워드 static으로 고쳤다. C는 대소문자를 구분한다.
- 4_1·4_2의 n_enter 앞에 unsigned를 추가했다. char는 유지하며 200을 셀 수 있는 범위를 명확히 했다.
- 4_3의 TCCR0 설정문 끝에 빠진 세미콜론을 추가했다.
- 4_4의 ISR 끝에 빠진 }를 추가하여 main을 독립된 함수로 만들었다.
- 4_4의 빈 배열은 10쪽 주파수표 및 11쪽 악보로 채웠다. 템포는 교재의 예시인 120 BPM을 적용했다.

검증 범위와 실행 전제

PDF 2쪽에 명시된 CPU 16MHz를 기준으로 계산했다. Timer0는 기본 동기 클록 상태(AS0 = 0)를 전제로 한다. 4_1–4_3은 분주비 1024, 4_4는 분주비 32다.

4_4의 _delay_ms를 위해 F_CPU를 16000000UL로 프로젝트 설정에 지정하고 실제 CPU 클록도 맞춘다. 원문에 없는 #define을 임의로 넣지 않았다. for문 안 변수 선언을 지원하는 C99/GNU99 이상으로 빌드한다.

원문 기호 보존, 배열 길이와 계산값, PDF의 전체 코드 수록 여부를 확인했다. AVR 컴파일과 실물 보드 실행은 하지 않았다. 원문 동작상 한계는 뒤쪽에 별도로 정리했다.

5ms와 약 1초를 만드는 계산

공통 타이머 클록

CPU 클록 = 16,000,000Hz, 분주비 N = 1024
타이머 한 카운트 시간 = N / f_clk = **64µs**
필요한 카운트 수 = 0.005 × 16,000,000 / 1024 = **78.125**

정수 카운트 78을 사용한다. 따라서 이상적인 타이머 간격은 78 × 64µs = **4.992ms**이고, 200회는 **998.4ms**다. 교재의 5ms와 1초는 근사 표현이다. ISR 지연과 재적재 시점은 실제 측정에 영향을 준다.

실습	설정/갱신 값	이유
4_1 오버플로	TCNT0 = 178	256 − 78 = 178
4_2 표준모드 비교일치	OCR0 = 78, 이후 OCR0 += 78	계속 증가하는 카운터에서 다음 비교 목표를 78만큼 이동
4_3 CTC	OCR0 = 77	0부터 77까지 78카운트이므로 OCR0 = 78 − 1

레지스터 값 암기

설정	의미
TCCR0 = 0x00	표준모드, 타이머 정지
TCCR0 |= 0x07	CS02:CS00 = 111 → 1024분주로 시작
(1<<WGM01) | (1<<COM00)	0x18 → CTC + OC0 비교일치 토글, 클록 정지
0x18에 0x07을 OR	0x1F → CTC + 토글 + 1024분주
TCCR0 = 0x03	CS02:CS00 = 011 → 표준모드, 32분주

4_2의 PDF 설명에는 “CTC”와 “오버플로”라는 표현이 섞여 있다. 실제 배포 코드는 TCCR0 = 0x00 및 TIMER0_COMP_vect이므로 **표준모드의 비교일치**다. 4_3이 CTC 비교일치 실습이다.

실습 4_1 · 표준모드 · 오버플로 인터럽트

배포 TXT 기준 · 빈칸과 명시된 최소 오류 보완

```
// 실습 4_1. 표준모드에서 타이머/카운터 오버플로 인터럽트 방법

#include <avr/io.h>
#include <avr/interrupt.h>

static unsigned char pattern[8] // LED 패턴 테이블
= {0xFE, 0xFD, 0xFB, 0xF7, 0xEF, 0xDF, 0xBF, 0x7F};

ISR(TIMER0_OVF_vect) // 타이머0 오버플로 인터럽트 서비스루틴
{
    static int  index=0;// 패턴 인덱스
    static unsigned char n_enter=0;// 인터럽트 횟수

    TCNT0 = 178;// 5msec 후에 인터럽트발생
    n_enter++;// 인터럽트 횟수 증가

    if(n_enter == 200)// 1000msec
    {
        n_enter=0;
        PORTA = pattern[index++];// 패턴이동
        if(index==8) index=0;
    }
}

int main()
{
    DDRA  = 0xFF;// A 포트를 출력으로 설정
    PORTA = 0xFF;// LED를 끈다.

    // 타이머/카운터 0 설정
    TCCR0 = 0x00;// 표준모드, 타이머 정지
    TCNT0 = 178;// 타이머 초기 값 설정

    // 인터럽트 설정
    TIMSK = (1<<TOIE0);// 타이머0 오버플로 인터럽트 허용
    sei();// 전역 인터럽트 허용

    TCCR0 |= 0x07;// 분주비 1024로 타이머 시작
    while(1);
}
```

TCNT0를 main과 ISR에서 모두 178로 채웠다. 256에 도달하면 오버플로가 발생하고, ISR에서 다시 초기값을 쓴다. 200번째 ISR에서 pattern[0]을 출력한 뒤 인덱스를 증가시킨다.

실습 4_2 · 표준모드 · 비교일치 인터럽트

배포 TXT 기준 · 빈칸과 명시된 최소 오류 보완

```
// 실습 4_2. 표준모드에서 출력 비교 일치 인터럽트 방법

#include <avr/io.h>
#include <avr/interrupt.h>

static unsigned char pattern[8] // LED 패턴 테이블
= {0xFE, 0xFD, 0xFB, 0xF7, 0xEF, 0xDF, 0xBF, 0x7F};

ISR(TIMER0_COMP_vect) // 비교일치 인터럽트 서비스루틴
{
    static int  index=0;// 패턴 인덱스
    static unsigned char n_enter=0;// 인터럽트 횟수

    OCR0 += 78;// 5msec 후에 인터럽트발생
    n_enter++;// 인터럽트 횟수 증가

    if(n_enter == 200)// 1000msec
    {
        n_enter=0;
        PORTA = pattern[index++];// 패턴이동
        if(index==8) index=0;
    }
}

int main()
{
    DDRA  = 0xFF;// A 포트를 출력으로 설정
    PORTA = 0xFF;// LED를 끈다.

    // 타이머/카운터 0 설정
    TCCR0 = 0x00;// 표준모드, 타이머 정지
    OCR0 = 78;// 타이머 초기 값 설정

    // 인터럽트 설정
    TIMSK = (1<<OCIE0);// 타이머0 비교일치 인터럽트 허용
    sei();// 전역 인터럽트 허용

    TCCR0 |= 0x07;// 분주비 1024로 타이머 시작
    while(1);
}
```

OCR0 += 78;의 +=를 유지했다. OCR0는 8비트라 256을 넘으면 순환하며 다음 비교 목표를 만든다. TCNT0는 표준모드에서 계속 증가하고 오버플로도 하지만, 여기서 허용한 ISR은 비교일치 ISR이다.

실습 4_3 · CTC 모드 · 비교일치와 OC0 토글

배포 TXT 기준 · 빈칸과 명시된 최소 오류 보완

```
// 실습 4_3. CTC 모드를 이용하는 방법

#include <avr/io.h>
#include <avr/interrupt.h>

static unsigned char pattern[8] // LED 패턴 테이블
= {0xFE, 0xFD, 0xFB, 0xF7, 0xEF, 0xDF, 0xBF, 0x7F};

ISR(TIMER0_COMP_vect)//비교일치 인터럽트 서비스 루틴(약 5ms주기)
{
    static int  index=0;// 현재 LED 패턴 인덱스
    static unsigned char n_enter=0;//인터럽트 누적 카운트(5ms단위)

    n_enter++;// 인터럽트 횟수 증가
    if(n_enter == 200)// 약 1초마다 (5ms × 200)
    {
        n_enter=0;
        PORTA = pattern[index++];// 다음 LED 패턴 출력
        if(index==8) index=0;
    }
}

int main()
{
    DDRA  = 0xFF;// A 포트를 출력으로 설정
    DDRB |= (1<<DDB0);// PB0(OC0)를 출력으로 설정
    PORTA = 0xFF;// LED를 끈다.

    // 타이머/카운터 0 설정 : CTC 모드, 타이머 정지, OC0핀 사용
    TCCR0 = (1<<WGM01) | (1<<COM00); // CTC 모드와 OC0 핀 토글 설정

    OCR0 = 77;// 5msec마다 비교일치 발생

    // 인터럽트 설정
    TIMSK = (1<<OCIE0);// 타이머0 비교일치 인터럽트 허용
    sei();              // 전역 인터럽트 허용

    TCCR0 |= 0x07;// 분주비 1024로 타이머 시작
    while(1);
}
```

CTC에서는 비교일치 후 카운터가 하드웨어에 의해 0으로 초기화된다. OCR0를 ISR에서 다시 쓰지 않는다. OC0는 PB4다. 배포 코드의 PB0 표기 문제는 뒤쪽 주의사항에 설명했다.

실습 4_4 · 음계 초기값 계산

왜 주기의 절반을 계산하나?

ISR에서 PB4를 HIGH와 LOW로 번갈아 바꾸므로 오버플로 한 번은 한 주기의 절반이다. Timer0의 한 틱은 32 / 16MHz = **2µs**다.

반주기 카운트 수 = f_clk / (2 × N × f_note)
TCNT0 = 256 − round(16,000,000 / (2 × 32 × f_note))

예: 도 1046.6Hz → 반주기 약 477.74µs → 약 239카운트 → TCNT0 = 256 − 239 = **17**. 교재의 첫 정답 17과 일치한다.

교재 음계명	목표 주파수(Hz)	TCNT0
도	1046.6	17
레	1174.6	43
미♭	1244.5	55
파	1397.0	77
솔	1568.0	97
라♭	1661.2	106
시♭	1864.7	122
높은 도	2093.2	137

가장 가까운 정수 카운트로 반올림했다. 높은 도 값 137은 표의 보충 답이며, 배포 코드 f_table[7]에는 앞의 7개만 들어간다. 배열 크기를 8로 바꾸지 않았다.

```
char f_table[7] = {17, 43, 55, 77, 97, 106, 122};
```

교재의 음계 표기 차이: #define MI 2이지만 표의 인덱스 2는 자연음 미가 아니라 미♭ 1244.5Hz다. 표의 다른 음도 라♭·시♭로 되어 있다. 이를 임의로 자연음 주파수로 교체하지 않았다. 악보의 자연음과 음정 차이가 생길 수 있으므로 시험에서는 배포 표를 기준으로 구분한다.

실제 주파수는 정수화 오차 외에도 ISR 진입·레지스터 재적재에 걸리는 시간의 영향을 받는다.

실습 4_4 · 전체 코드 (1/2)

배포 TXT 기준 · 음계·악보 배열 및 ISR / 다음 쪽 main으로 이어짐

```
// 실습 4_4.c  /* '비행기' 연주하기 */

#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

// 음표 상수 정의
#define DO 0            // '도' 음을 나타내는 상수
#define RE 1    // '레' 음을 나타내는 상수
#define MI 2            // '미' 음을 나타내는 상수

#define EOS -1          // 노래의 끝을 나타내는 상수

// 각 음의 주기 값 (타이머 초기화 값)
char f_table[7] = {17, 43, 55, 77, 97, 106, 122};

// 비행기 노래의 음표 배열 (계이름에 맞게 구성)
int song[] = {
    // (끝에 EOS)
    MI, RE, DO, RE,
    MI, MI, MI,
    RE, RE, RE,
    MI, MI, MI,
    MI, RE, DO, RE,
    MI, MI, MI,
    RE, RE, MI, RE,
    DO, EOS
};

// 각 음의 지속 시간 (밀리초)
int note_duration[] = {
    // (마지막은 0)
    750, 250, 500, 500,
    500, 500, 1000,
    500, 500, 1000,
    500, 500, 1000,
    750, 250, 500, 500,
    500, 500, 1000,
    500, 500, 500, 500,
    2000, 0
};

int state = 0; // 현재 음의 ON/OFF 상태를 나타내는 변수
int tone = 0;  // 현재 재생 중인 음표를 나타내는 변수

// 타이머/카운터0 오버플로우 인터럽트 서비스 루틴
ISR(TIMER0_OVF_vect)
{
    if (state == 1)
    {
        PORTB = 0x00; // PORTB 핀 설정: 소리 OFF
        state = 0;   // 상태를 OFF로 설정
    }
    else // state가 0이면 소리를 ON 상태로 전환
    {
        PORTB = 0x10; // PORTB 핀 설정: 소리 ON
        state = 1;    // 상태를 ON으로 설정
    }
    TCNT0 = f_table[tone];
}
```

실습 4_4 · 전체 코드 (2/2)

앞쪽 코드에 이어지는 main 함수 · 같은 C 파일에 연속해서 작성

```
int main()
{
    int i = 0;
    DDRB = 0x10;
    TCCR0 = 0x03;
    TIMSK = (1 << TOIE0);
    sei();

    while (song[i] != EOS)
    {
        tone = song[i];
        TCNT0 = f_table[tone];
        PORTB = 0x10;

        int delay_time = note_duration[i];
        for (int j = 0; j < delay_time / 10; j++)
        {
            _delay_ms(10);
        }

        PORTB = 0x00;
        _delay_ms(100);
        i++;
    }
    return 0;
}
```

배포본에서 누락된 ISR의 닫는 중괄호를 앞쪽 마지막에 추가했다. main 안의 연주 순서는 그대로 유지했다. 100ms 동안 인터럽트가 정지하지 않는 점과 공유 변수 관련 주의사항은 뒤쪽 해설을 참고한다.

악보와 음표 길이 읽기

120 BPM 예시 적용

한 박자 = 60,000 / BPM = 60,000 / 120 = **500ms**
점4분음표 = 750ms, 8분음표 = 250ms
4분음표 = 500ms, 2분음표 = 1000ms, 온음표 = 2000ms

PDF 11쪽 악보의 첫 음은 점4분음표, 둘째 음은 8분음표다. 모두 같은 500ms로 채우지 않았다. 음높이 기호는 배포 코드의 DO, RE, MI를 사용했다.

마디	song 배열	note_duration (ms)
1	MI, RE, DO, RE	750, 250, 500, 500
2	MI, MI, MI	500, 500, 1000
3	RE, RE, RE	500, 500, 1000
4	MI, MI, MI	500, 500, 1000
5	MI, RE, DO, RE	750, 250, 500, 500
6	MI, MI, MI	500, 500, 1000
7	RE, RE, MI, RE	500, 500, 500, 500
8	DO	2000

실제 음표는 25개이며 song 끝에 EOS, note_duration 끝에 0을 추가하여 두 배열은 각각 26개다. EOS는 종료 표시이므로 f_table의 인덱스로 사용되지 않는다.

main의 재생 순서

1. song[i]가 EOS인지 확인한다.
2. tone과 TCNT0를 설정한다.
3. note_duration[i] / 10회만큼 _delay_ms(10)을 반복한다.
4. PORTB에 0을 쓰고 100ms 대기한다.
5. i를 증가시켜 다음 음으로 이동한다.

> 음표 길이 합은 16,000ms다. 배포 코드는 각 음 뒤에 100ms를 추가하므로 전체 지연문 합은 18,500ms이며, 이는 악보의 정확한 120 BPM 연주 길이와 다르다. ISR 실행 시간도 실제 경과시간에 더해진다.

교재 코드에서 반드시 구분할 점

1. 4_3의 PB0(OC0) 주석

```
DDRB |= (1<<DDB0);// PB0(OC0)를 출력으로 설정
```

배포 문장을 그대로 유지했다. 하지만 첨부 ATmega128 데이터시트에서 **OC0는 PB4**다. PB0는 OC0 출력 핀이 아니다. 따라서 이 줄 그대로는 PB4의 하드웨어 토글 파형을 출력하도록 방향을 설정하지 못한다. LED의 PORTA 출력은 별개로 진행할 수 있다.

> 이 자료는 DDB0를 DDB4로 몰래 교체하지 않았다. OC0 파형을 실측하는 과제라면 교수자의 핀 수정 지시를 확인해야 한다. 데이터시트 기준 수정 대상은 DDB0 → DDB4다.

2. 4_4의 100ms는 완전한 무음이 아니다

```
PORTB = 0x00;
_delay_ms(100);
```

main이 한 번 LOW를 써도 Timer0 인터럽트는 계속 실행되어 PB4를 다시 HIGH/LOW로 바꾼다. 따라서 배포 코드의 “음 사이 100ms”를 확실한 휴지 구간이라고 설명하면 안 된다. 타이머 정지나 출력 차단 로직은 원문 실행 흐름을 바꾸므로 추가하지 않았다.

3. 마지막은 무한 반복이 아니라 return

PDF의 개요와 달리 배포 main은 while(song[i] != EOS)를 빠져나온 뒤 return 0;에 도달한다. 노래를 처음부터 반복하는 코드는 없다. 복귀 후 최종 상태는 AVR 런타임의 종료 처리에 따라 달라진다.

4. 소리 ON/OFF 주석의 뜻

ISR의 PORTB = 0x10과 0x00은 파형의 HIGH/LOW 두 반주기다. 각각을 “음을 시작함/멈춤”으로 해석하지 않는다. TCCR0 = 0x03은 표준모드이며, 하드웨어 PWM 모드가 아니라 ISR로 핀을 토글하는 소프트웨어 사각파 생성이다.

5. main과 ISR이 공유하는 tone

배포본의 int tone은 유지했다. 최적화 환경에서는 공유 변수의 volatile 지정과 16비트 접근의 원자성도 검토 대상이다. 이 자료는 빈칸 중심으로 보충한 것으로, 최적화 조건까지 포함한 완전한 동작 보증본은 아니다.

시험 직전 빈칸 정답과 비교

4_1·4_2 빈칸 순서

번호	4_1 오버플로	4_2 비교일치
① ISR 벡터	TIMER0_OVF_vect	TIMER0_COMP_vect
② 초기값/증가값	178 (0xB2)	78 (0x4E)
③ 인터럽트 횟수	200	200
④ TIMSK 비트	TOIE0	OCIE0
⑤ 분주 설정	0x07	0x07

4_3 빈칸 순서

번호	채울 내용
①	TIMER0_COMP_vect
②	200
③	(1<<WGM01) | (1<<COM00); = 0x18
④	77 (0x4D)
⑤	OCIE0
⑥	0x07

반드시 구분할 기호

- 4_1은 TCNT0 = 178로 매번 재적재한다.
- 4_2는 OCR0 += 78로 다음 비교 목표를 이동한다.
- 4_3은 OCR0 = 77을 main에서 한 번 설정한다.
- pattern[index++]는 현재 인덱스로 출력한 뒤 증가한다. ++index로 바꾸지 않는다.
- while(1);의 세미콜론은 빈 반복문을 의미한다.

출처: 업로드한 05주차 PDF 2–3, 5–8, 9–12쪽과 배포 TXT 4개. 레지스터 검토: 첨부 ATmega128L 데이터시트의 Timer0 모드·클록 선택표 및 PB4/OC0 핀 설명.